

Image Pair Classifier

Convolutional Neural Networks in Practice

Sean Bush

This project seeks to illustrate the process of creating a convolutional neural network for the purpose of image processing and computer vision. The purpose of this model is to examine a pair of images and determine whether they come from the same source image, despite differences in scale, cropping, color, or artifacts. To this end, the library PyTorch is used to create a convolutional neural network with a Siamese architecture. The network is then trained on the CIFAR100 dataset and tuned using a randomized hyperparameter search. The resulting model classifies unseen image pairs from the CIFAR100 dataset with 97.93% accuracy.

Business Problem

While this project is purely a hypothetical exercise, not tied to any individual organization or specific business problem, the techniques I employ within are applicable to many different types of problems a business may face. For example, one of the primary hypotheticals guiding this project was that of a reverse image searching algorithm. Reverse image searching is a service provided by web companies like Google, TinEye, and Yandex. You upload an image file, and it scours the web for images that match it, even if there are differences in resolution or scale, or if there is artifacting caused by image compression or screenshotting. These websites have to employ some model capable of matching if two images, despite their differences, come from the same source image. In this project, I seek to create such a model.

Dataset Description

This project required a large dataset containing images of many different kinds of objects. I decided that a common, low-resolution dataset with many examples would be appropriate to prevent training times from becoming prohibitively large. As such, I selected the CIFAR100 dataset, a dataset of sixty thousand images, each given one of one hundred class labels (Krizhevsky, 2009). Each image in the dataset has a resolution of 32 x 32 pixels (Krizhevsky, 2009). The data is also split into one training set containing fifty thousand of the images, and one testing set containing the remaining ten thousand (Krizhevsky, 2009). The split is stratified along class lines such that the proportion of images belonging to each set is equivalent for the training and testing sets (Krizhevsky, 2009).

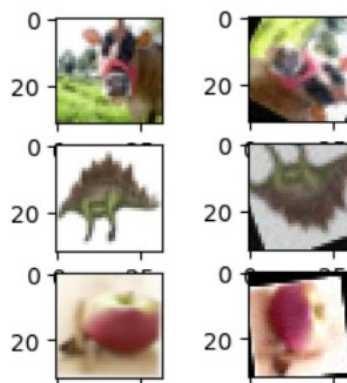
This analysis need not make use of the class labels, but the large number of labels in CIFAR100 ensures that the model will see examples of many different kinds of objects, making it preferable to its more well-known sibling CIFAR10. The other major advantage of using this dataset is that it is built into the Python library torchvision, whose data augmentation methods this project uses heavily (Torchvision developers, n.d.-a). As such, installing the dataset and loading it for use is easier than having to download a set from an external source. It should be mentioned that in a real-world context, you would want a higher-resolution and more varied dataset to train your model than CIFAR, but for the academic context of this project, the lower resolution and easily accessible CIFAR dataset seemed more appropriate.

Data Preparation

The torchvision library provides the CIFAR100 dataset as one of its Dataset objects, as stated previously. Each item in this dataset is a tuple containing an image in the PIL format defined by the Python library pillow (Pillow developers, n.d.) and the class label as a number between 1 and 100 (Torchvision developers, n.d.-a). The end goal of the data preparation process is to convert this into a dataset where each item is a tuple containing two different images and a label for whether or not those images originate from the same source.

The first step to prepare this data is to determine which modifications should be made to generate child images from the same parent image. I used the example business problem of a reverse image searching algorithm to guide the selection. The following transformations are identified as plausible ways an image might be modified as it is passed around online: rotation, translation, scaling, flipping, blurring, noise, and color jitter affecting the hue, saturation, brightness, and contrast of an image.

The torchvision library provides a transforms submodule that simulates each of these transformations. In the script transform.py, accessible from this project's GitHub repository, I composed the aforementioned transforms using such torchvision classes as RandomAffine, RandomHorizontalFlip, and ColorJitter (Torchvision developers, n.d.-b). The effects of these transformations on the images can be previewed in the Jupyter Notebook preview_transforms.ipynb, also accessible from the project's repository. A sample of the transformations is shown below.



Once the transformations were decided upon, I had to determine when they would be applied. One option is to apply the transformation before training, essentially creating a new dataset of image pairs. This option could be called “baking” the transformation. The second option is to do the transformations dynamically at runtime. The advantage of the latter option is that it creates a significantly larger number of image pairs. While it can’t increase the number of source images, it can be nearly guaranteed that the same exact pair of images will never be seen more than once, which would give the model many more training examples to learn from and prevent overfitting.

However, the dynamic method comes with a significant disadvantage: performing these transformations at runtime is expensive. When creating the code for this project, this was the option I originally used to train the model. It resulted in a single training epoch taking more than a minute to train on my personal computer. These times would have been prohibitively slow for performing techniques like hyperparameter tuning, which trains across many training epochs. Therefore, I determined that the slightly less effective but significantly more efficient baking method would be better suited to this analysis.

The baking process is performed by the script `preprocess_data.py`. The first thing this script does is load in the default transformation defined by `transform.py` and download the CIFAR100 dataset using `torchvision`.

```
print('Loading data...')
transform = get_transform()
os.makedirs('cifar', exist_ok=True)
train_and_validate = CIFAR100(root='cifar', train=True, download=True, transform=transform)
test = CIFAR100(root='cifar', train=False, download=True, transform=transform)
print('Complete.')
```

Next, it splits the datasets into three sections. The CIFAR100 dataset is already split into a training and a testing set. For this analysis, I wanted to move a number of the training examples into a third validation set. The script takes the first forty thousand examples and stores them as the new training set, and the remaining ten thousand as the new validation set. The testing set is unchanged.

```
print('\nRendering training data...')
data['train_firsts'], data['train_seconds'], data['train_targets'] = _render(train_and_validate, 0, 40000, random_state=0)
print('Complete')

print('\nRendering validation data...')
data['validate_firsts'], data['validate_seconds'], data['validate_targets'] = _render(train_and_validate, 40000, 50000, random_state=1)
print('Complete.')

print('\nRendering testing data...')
data['test_firsts'], data['test_seconds'], data['test_targets'] = _render(test, 0, len(test), random_state=2)
print('Complete.')
```

For each set, the helper function `_render()` is called, which randomly selects half of the images to be used in equal pairs and half to be used in unequal pairs. For the equal pairs, the parent image would be used to generate two different child images, and they would be returned along with a target value of 1.0, indicating that they come from the same source. For the unequal pairs, the image would be paired with a randomly chosen different image, with one child image being created from each. This pair is

returned with a target value of 0.0, indicating that they came from different source images. Lastly, the resulting images are saved into nine different numpy array files, three for each dataset. One of these three contains the first images of each pair; another contains the second images for each pair; and the third contains the target for each pair.

```
print('\nSaving data...')
os.makedirs('../data', exist_ok=True)
for name in data:
    array = data[name]
    np.save(f'../data/{name}.npy', array, allow_pickle=False)
print('Complete.')
```

When this script is run, the resulting dataset is saved to a directory called “data” to be used in the analysis.

This analysis uses a custom PyTorch Dataset object called ImagePairDataset, which is defined in the script data.py. When constructing an ImagePairDataset, the user must pass in a string stating whether they want to load the training, validation, or testing data. Then, the object will load the relevant numpy files from the data directory. Rather than loading the entire dataset into memory at once, it uses numpy’s “mmap” functionality to load a memory map of the array (NumPy developers, n.d.), allowing each individual training example to be loaded from disk dynamically. This prevents memory from being overloaded by storing the entire dataset at once. When the user requests an example from the ImagePairDataset, it loads the relevant images and target label from these files, converts each from a numpy array into a PyTorch tensor, and returns them as a tuple.

```
class ImagePairDataset(torch.utils.data.Dataset):

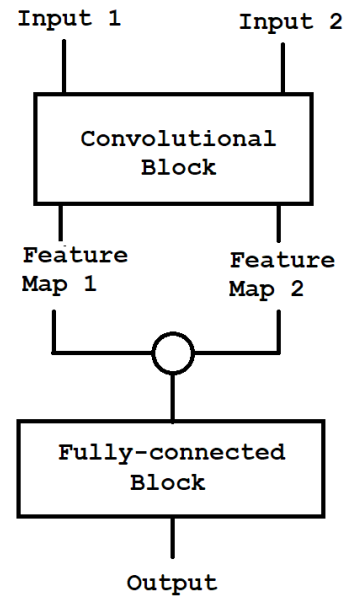
    def __init__(self, split: str):
        if split not in ['train', 'validate', 'test']:
            raise Exception('"split" parameter must be either "train", "validate", or "test"')
        self.firsts = np.load(f'data/{split}_firsts.npy', mmap_mode='r')
        self.seconds = np.load(f'data/{split}_seconds.npy', mmap_mode='r')
        self.targets = np.load(f'data/{split}_targets.npy', mmap_mode='r')

    def __getitem__(self, index: int) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        return torch.tensor(self.firsts[index]), torch.tensor(self.seconds[index]), torch.tensor(self.targets[index])

    def __len__(self) -> int:
        return self.targets.shape[0]
```

Model Architecture

At a high level, the model used in this analysis is what would be called a Siamese model. There is one set of convolutional layers that turns a single image into a set of features, and both images in each input pair are run through this same layer. This means the weights and biases responsible for transforming the first image are the same weights and biases responsible for transforming the second image (Dutt, 2025). While a common Siamese model might stop there, using some loss function designed especially for comparing two different features (Dutt, 2026), this project combines the results of the convolutional layers into a single vector and passes it through a fully-connected block that eventually outputs a single value. This output value represents the model's confidence that the images are the same. By default, the model outputs a logit rather than a scaled value that can be used for classification, so before the model output can be used, it must be run through a sigmoid activation function to map the possible output values to a range of zero to one.



The implementation of this model is contained in a class called `ImagePairMatcher`, accessible from the `model.py` script. As it is implemented, the model can be given any number of convolutional or fully-connected layers to implement at construction, but this analysis only ever uses three convolutional layers and three fully-connected layers.

```
@staticmethod
def from_config(config: Config) -> Self:
    model = ImagePairMatcher(cnn_hidden_layers=[config.CNN_HIDDEN_CHANNELS_1, config.CNN_HIDDEN_CHANNELS_2, config.CNN_OUT_CHANNELS],
                             fc_hidden_layers=[config.FC_HIDDEN_1, config.FC_HIDDEN_2],
                             cnn_dropout=config.CNN_DROPOUT,
                             fc_dropout=config.FC_DROPOUT)

    return model
```

Each convolutional layer is accompanied by a batch normalization layer, a ReLU layer, a max pooling layer, and a dropout layer. Though the dropout layer's effects can be optionally disabled by setting the model's `cnn_dropout` parameter to zero. After an image is run through the convolutional block, it is then put through a 2D global average pooling layer, which takes the average value of each feature map. The resulting averages are then flattened into a 1D feature map tensor.

```

cnn_blocks = []
cnn_hidden_layers = [3] + cnn_hidden_layers
for layer_in, layer_out in zip(cnn_hidden_layers[:-1], cnn_hidden_layers[1:]):
    cnn_block = nn.Sequential(
        nn.Conv2d(in_channels=layer_in, out_channels = layer_out, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(num_features=layer_out),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),
        nn.Dropout2d(cnn_dropout)
    )
    cnn_blocks.append(cnn_block)
self.cnn = nn.Sequential(*cnn_blocks)

self.gap = nn.AdaptiveAvgPool2d(output_size=(1, 1))
self.flat = nn.Flatten()

```

Both images in each input pair are put through the same convolutional block. After this, they are combined into a single input. To give the model as much information as possible, this combined input contains the raw features for each input, along with the absolute difference between the features and the Hadamard product of the features.

```

def _embed(self, x: Tensor) -> Tensor:
    x = self.cnn(x)
    x = self.gap(x)
    return self.flat(x)

def forward(self, x1: Tensor, x2: Tensor) -> Tensor:
    features1 = self._embed(x1)
    features2 = self._embed(x2)
    difference = torch.abs(features1 - features2)
    hadamard = features1 * features2
    x = torch.cat([features1, features2, difference, hadamard], dim=-1)
    x = self.fc(x)
    return self.out(x).squeeze(-1)

```

This combined input is then run through the fully-connected layers, each one consisting of a standard linear layer, a ReLU layer, and an optional dropout layer. The final layer of the model is another linear layer, unaccompanied by any activation function or dropout layers, which reduces the output to a single value: the model's output logit.

```

fc_blocks = []
fc_hidden_layers = [4 * cnn_hidden_layers[-1]] + fc_hidden_layers
for layer_in, layer_out in zip(fc_hidden_layers[:-1], fc_hidden_layers[1:]):
    fc_block = nn.Sequential(
        nn.Linear(layer_in, layer_out),
        nn.ReLU(),
        nn.Dropout(fc_dropout)
    )
    fc_blocks.append(fc_block)
self.fc = nn.Sequential(*fc_blocks)

self.out = nn.Linear(fc_hidden_layers[-1], 1)

```

The ImagePairMatcher dataset provides the following methods for interacting with this network:

- `_embed()` - a function that takes in a single image output, puts it through the convolutional layers, and outputs the feature vector
- `forward()` - the function used for defining the model's default functionality according to the normal PyTorch workflow. Calls `_embed()` on both input images, combines the output features into a single tensor, runs that tensor through the fully-connected layers, and returns the output value.
- `predict()` - a function that calls `forward()` on a pair of input images and wraps the output in a sigmoid function to clamp the value between zero and one. The function returns True if the value is greater than or equal to the supplied threshold (0.5 by default) or False otherwise.
- `parameter_count()` - a method to supply the number of parameters in the model that are trainable
- `from_config()` - a static method that constructs an ImagePairMatcher based on a supplied Config object (the Config class will be explained in the next section)

Model Training

Model training was performed using the Adam optimizer and a Binary Cross-Entropy loss function. These were chosen as sensible default options for model training and were not altered, as the model performance they yielded was sufficient. The training program was designed from the ground up to support hyperparameter tuning. This meant that I needed to create a system that could take in a parameter grid and automatically alter the model architecture and training behavior accordingly. This is the purpose behind the Config class, a Python data class (Python developers, n.d.) used to contain all of the model's hyperparameters and to specify default values for each. The class definition can be found in the `config.py` script.

```

@dataclass
class Config:

    # Model Architecture
    CNN_HIDDEN_CHANNELS_1: int = 16
    CNN_HIDDEN_CHANNELS_2: int = 32
    CNN_OUT_CHANNELS: int = 64
    CNN_DROPOUT: float = 0.0

    FC_HIDDEN_1: int = 32
    FC_HIDDEN_2: int = 16
    FC_DROPOUT: float = 0.0

    # Training
    NUM_EPOCHS: int = 20

    LEARNING_RATE: float = 0.001
    WEIGHT_DECAY: float = 0.0

    PATIENCE: Optional[int] = None
    DELTA: Optional[float] = None

```

A Config object allows the user to specify the number of channels for each convolutional layer, the number of neurons in each fully-connected layer; dropout values for the convolutional and fully-connected layers; the number of training epochs to perform; the learning rate and weight decay to be used with the optimizer; and patience and delta values to define early stopping behavior during training. The class also has a static method called `generate_randomized()` that is used to enable a randomized hyperparameter search. The user passes in a dictionary with the name of each hyperparameter as the key, and a list of possible values for each. Then, the function chooses random values from each list and uses them to create a new Config object.

```

@staticmethod
def generate_randomized(param_grid: dict[str, ArrayLike], random_state: Optional[int] = None) -> Self:
    config = Config()
    rng = Random(random_state)
    for hyperparameter, options in param_grid.items():
        if hyperparameter not in list(filter(lambda s: s[0] != '_', dir(config))):
            raise ValueError(f'{hyperparameter} is not a valid hyperparameter')
        setattr(config, hyperparameter, rng.choice(options))
    return config

```

Model training behavior is defined in the `train.py` script. The code in this script is often too long to duplicate here in full, but snippets will still be provided where applicable. The `train()` function is responsible for training a model. It takes a model, a loss function, an optimizer, the number of epochs, data loaders for the training and validation sets, the device to run the analysis on (to enable GPU acceleration), an early stopping object which stores the early stopping parameters, and a boolean flag to determine if a progress bar should be displayed. It then uses those supplied objects to perform a training loop over the supplied number of epochs, alternating between training the model on an epoch of training data and evaluating it on an epoch of validation data.


```
def train(model: Module,
          criterion: Loss,
          optimizer: Optimizer,
          num_epochs: int,
          train_loader: DataLoader,
          validate_loader: DataLoader,
          device: Device,
          early_stop: Optional[EarlyStopper] = None,
          progress: bool = True) -> TrainResults:
```

This function tracks and stores the loss and accuracy values of the model at the end of each epoch in a custom data class called TrainResults, and then it returns it when training is complete.

```
@dataclass
class TrainResults:
    train_losses: list[float]
    train_accuracies: list[float]
    validate_losses: list[float]
    validate_accuracies: list[float]
```

The test() method is similar except that it only needs a model, a data loader for the testing set, a device, and a loss function. It returns the loss and accuracy of the model on the testing set as a custom TestResults object.

```
@dataclass
class TestResults:
    test_loss: float
    test_accuracy: float
```

The run_pipeline() method automates the entire process of model creation and training based on the parameters in a supplied Config. It gives the best high-level view of the model training process used in this project. It instantiates an ImagePairMatcher model; instantiates the Adam optimizer, specifying learning rate and weight decay values from the Config; loads the device used for connecting to the GPU; loads the training and validation sets, creating data loaders for each; calls train() with all of the aforementioned objects; and finally returns the TrainResults.

```

def run_pipeline(config: Config, progress: bool = True) -> TrainResults:

    model = ImagePairMatcher.from_config(config)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(),
                            lr=config.LEARNING_RATE,
                            weight_decay=config.WEIGHT_DECAY)
    device = torch.device('cuda')
    early_stop = EarlyStopper.from_config(config)

    train_data = ImagePairDataset(split='train')
    validate_data = ImagePairDataset(split='validate')

    train_loader = DataLoader(train_data, batch_size=128, shuffle=True, num_workers=4, persistent_workers=True)
    validate_loader = DataLoader(validate_data, batch_size=128, num_workers=4, persistent_workers=True)

    train_results = train(model, criterion, optimizer, config.NUM_EPOCHS, train_loader, validate_loader, device, early_stop, progress)

    return PipelineResults(train_results, model, criterion, optimizer, device, early_stop)

```

The last function in the train.py script is randomized_search(). It takes in the number of models to be trained and a parameter grid dictionary, which must be compatible with the Config class's generate_randomized() method. It then creates the models with hyperparameters randomly chosen from the parameter grid, trains each using the run_pipeline() method, and records the final validation loss for each model. Lastly, it outputs the best performing model along with every model's Config and scores.

```

def randomized_search(param_grid: dict[str, ArrayLike], n: int) -> TuneResults:

    best_config = None
    best_model = None
    best_score = None
    best_train_results = None
    scores = {}

    for _ in tqdm(range(n)):

        config = Config.generate_randomized(param_grid)
        results = run_pipeline(config, progress=False)

        model = results.model
        train_results = results.train_results
        score = train_results.validate_losses[-1]

        scores[str(config)] = score

        if best_score is not None and score >= best_score:
            continue

        best_config = config
        best_model = model
        best_score = score
        best_train_results = train_results

    return TuneResults(best_config, best_model, best_score, train_results, scores)

```

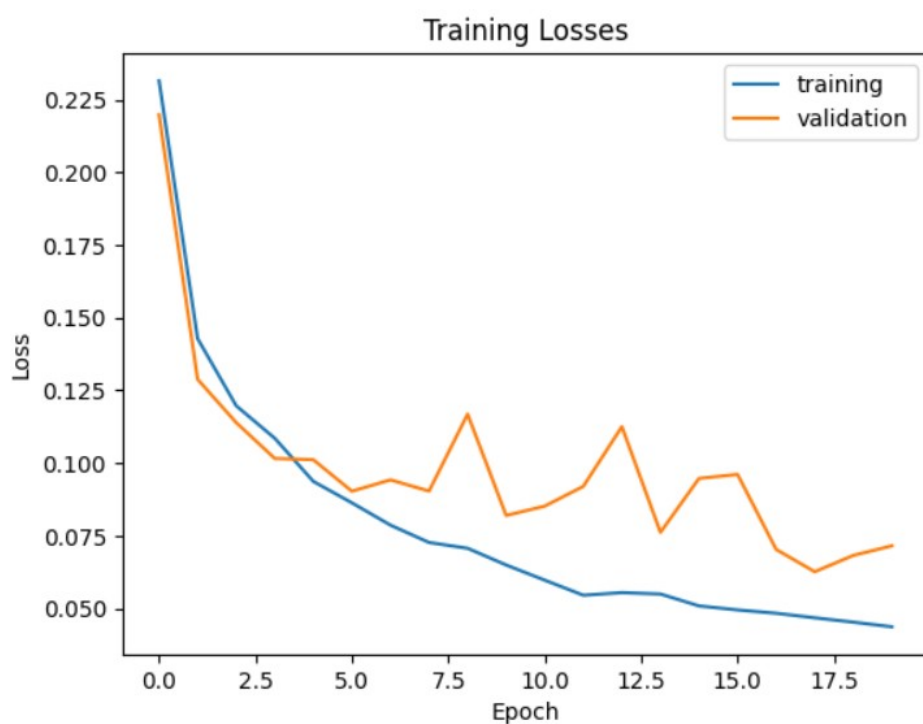
The code that utilizes all of these functions to perform the actual analysis for this project exists within the analysis.ipynb notebook file. The following parameter grid was used for the hyperparameter search:

```
param_grid = {  
    'CNN_DROPOUT': np.linspace(0.0, 0.3, num=4),  
    'FC_DROPOUT': np.linspace(0.0, 0.3, num=4),  
    'LEARNING_RATE': np.logspace(-4, -2, num=20),  
    'WEIGHT_DECAY': np.logspace(-4, -2, num=7)  
}
```

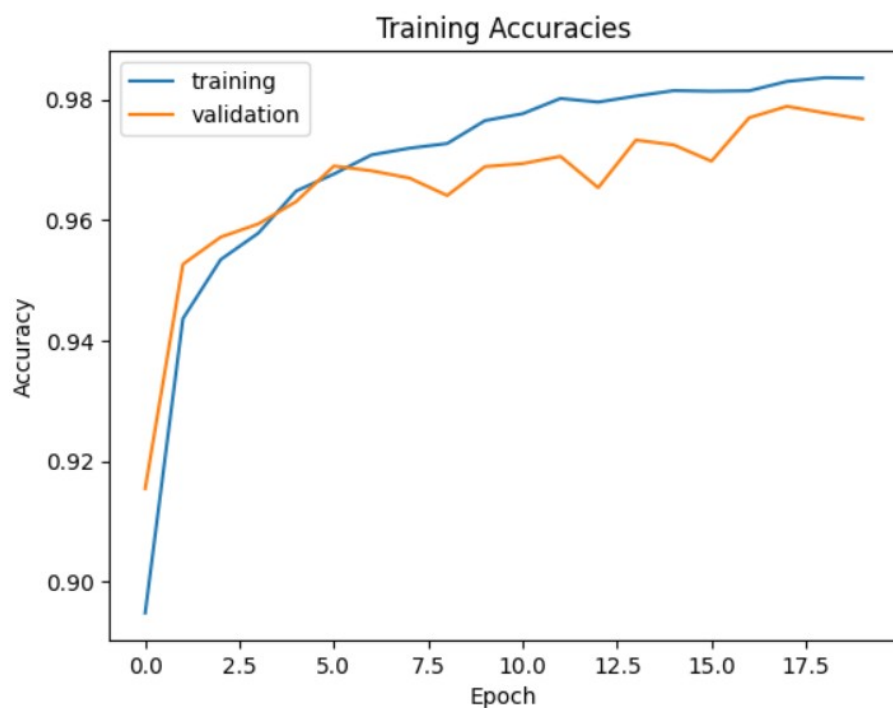
Note that the grid did not utilize every hyperparameter available in the Config object. One of the benefits of making the Config a data class is that it uses default values for the other parameters. If I were unhappy with the model the hyperparameter search found, I could expand the hyperparameter grid to include these other values. As will be explained in the next section, it was not necessary to get a well-performing model.

Results

The randomized search was performed with one hundred different models and found that the optimal set of hyperparameters was as follows: the dropout proportions for the convolutional layers and the fully-connected layers were zero; the learning rate for the optimizer was about 0.0023; the weight decay for the optimizer was about 0.0002. The resulting model managed to get an accuracy of 98.36% on the training data, 97.68% on the validation data, and 97.93% on the testing data. The following graph shows the loss curves of the model during training:



The next graph shows the model accuracy during training.



It is clear from the metrics and graphs that there is a tiny amount of overfitting occurring within the model. It seems to perform consistently better on seen data than on unseen data. However, it is even clearer that the model is, overall, extremely effective. Within the first epoch of training, it was able to

achieve a near 90% accuracy on training examples, and the final testing accuracy of nearly 98% is about as good as one could hope for.

Limitations

This project has a number of limitations, several of which are likely responsible for inflating the accuracy numbers beyond what you would expect in a real-world implementation of this model architecture. The first is the fairly narrow set of transformations being applied to the raw images in this dataset. While care was taken to ensure the images in the dataset were being altered in many different ways, there are still several obvious omissions that would be necessary to account for if the model were being trained for use in a real-world context. The most blatant omission is repeated image compression, the kind that would come from taking a screenshot or converting an image to a different file type. Another significant omission is the lack of variance in image resolutions. In a real-world context, you would not expect an image that has been screenshotted, cropped, or scaled to have the same resolution as the source image. It would be even less expected for every image to have the exact same resolution, as they do in this dataset. In this way, the choice of CIFAR for the data is unrepresentative of the real-world context in which one might use this kind of model. While using a low-resolution, easily-accessible dataset was helpful for this purely academic context, it does pose a significant limitation for its generalizability.

Another key limitation to note is the relative lack of exploration of different kinds of model architectures or training methodologies. That is to say, the grid of hyperparameters that was searched over while tuning the model was fairly small. This project did not utilize tuning the early stopping parameters, nor the ability to alter the number of features or neurons in the convolutional and fully-connected blocks. It did not consider other optimizing algorithms besides Adam. When evaluating the different models, it did not consider a variety of model fit metrics, only the final validation loss. In essence, the hyperparameter search was far too limited in scope to grant confidence that the model is nearly as effective as it could be. In a similar vein, the metrics used for model testing were quite limited. The project only considered model loss and accuracy, ignoring other classification metrics such as recall, precision, and F1 score.

Next Steps

If one desires to improve upon this project, addressing its limitations and expanding its scope, the most obvious first step to take is to create a more effective dataset. This would include images at a significantly higher resolution. Then, the transformation of image pairs would include some transformations noted as missing in the limitations section, namely, image compression and resolution alterations. Notably, the model architecture, as it appears in this project, is capable of handling images of any resolution as is. The feature is simply not utilized in the present analysis.

Another step would be to address the lack of scope in hyperparameter tuning and model evaluation. This would include expanding the number of hyperparameters to be tuned, including

alterations to model architecture and optimizer algorithm, and an expansion of diagnostic criteria. Doing so would expand the search space significantly and increase the likelihood of finding an optimal model, at the expense of significantly more compute power being required.

Clearly, this project is ripe for continuation. The primary concern an analyst would need to consider is computer power. Convolutional neural networks are not trivial to train, and my personal computer used to train this model was only sufficient to handle low-resolution images and a fairly narrow hyperparameter search space. Also, it was unable to handle image transformation during training, making it necessary to bake them beforehand. With a more powerful machine and minimal changes to the existing code, this project could be improved significantly from one with purely academic worth to one useful in the real-world context of reverse image-searching.

References

- Dutt, A. (2025, March 5). *Siamese networks introduction and implementation*. Towards Data Science. <https://towardsdatascience.com/siamese-networks-introduction-and-implementation-2140e3443dee/>
- Krizhevsky, A. (2009, April 8). Learning Multiple Layers of Features from Tiny Images. University of Toronto. <https://cave.cs.toronto.edu/kriz/cifar.html>
- NumPy developers. (n.d.). *Numpy.load*. NumPy v2.4 Manual. <https://numpy.org/doc/stable/reference/generated/numpy.load.html>
- Pillow developers. (n.d.). *Pillow*. PyPI. <https://pypi.org/project/pillow/>
- Python developers. (n.d.). *Dataclasses - Data Classes*. Python documentation. <https://docs.python.org/3/library/dataclasses.html>
- Torchvision developers. (n.d.-a). *CIFAR100*. torchvision 0.27 documentation. <https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.CIFAR100.html#torchvision.datasets.CIFAR100>
- Torchvision developers. (n.d.-b). *Transforming Images, videos, boxes and more*. Transforming images, videos, boxes and more - torchvision 0.27 documentation. <https://docs.pytorch.org/vision/stable/transforms.html>